

410 Presentation

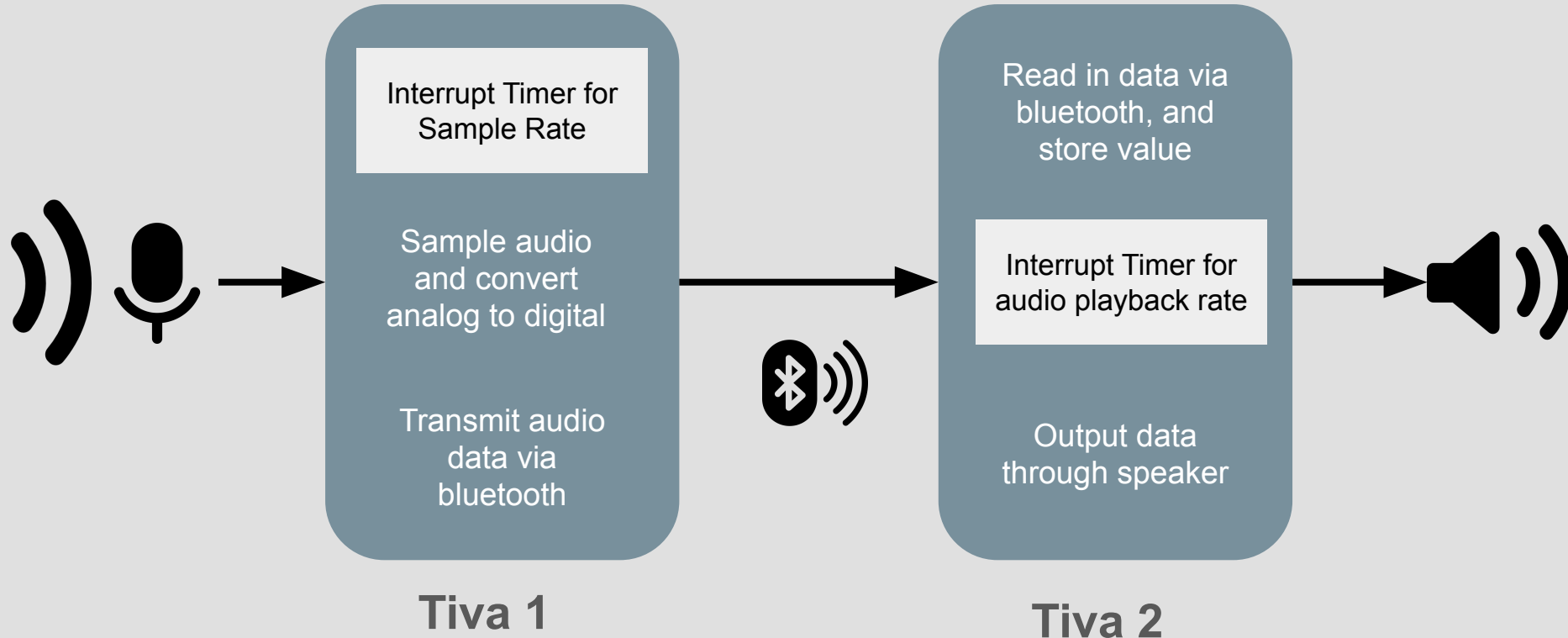
Walkie - Talkie

...

Jordan and Timothy

Project Description (Original goal)

Not exactly what happens



Main Aspects

Record Audio

In order to record audio, you need to:

- Convert audio waves into an electric signal
- Process those electric signals
- Convert analog data to digital data

Data Transmission

For real time data transmission you need to:

- Be able to transmit data at a high enough speed
- Make sure data isn't lost during transmission

Audio Output

Convert digital to analog values, whilst keeping the original frequency.

Recording Audio:
Pain, pain and more pain

Recording Audio data

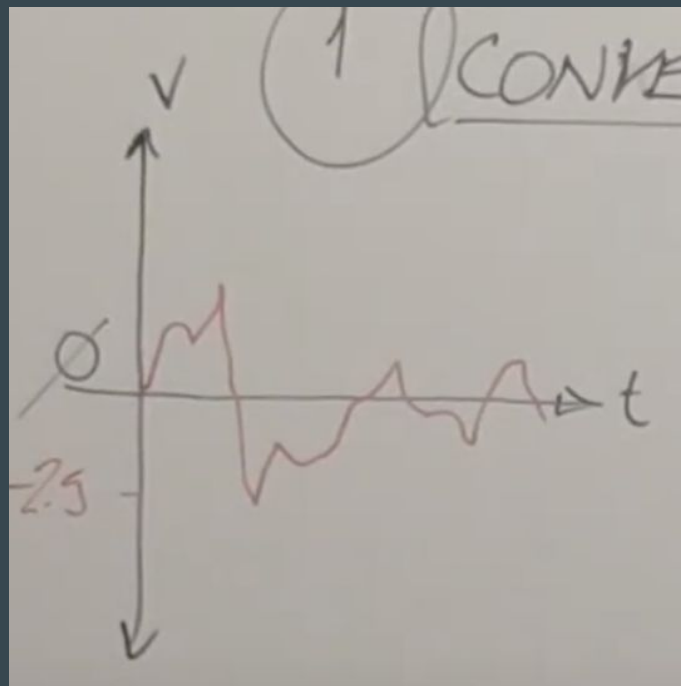
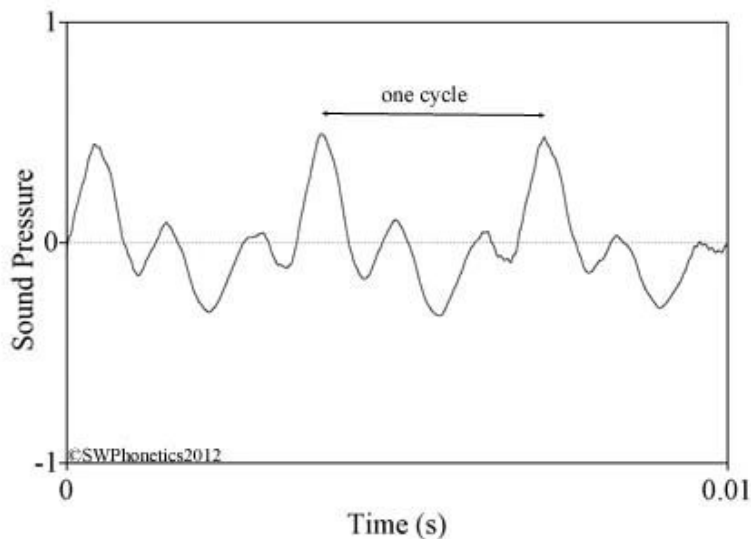
A microphone takes in an audio wave and converts it to a voltage signal



A sound wave is a pressure wave consisting of areas of high pressure and areas of low pressures. An electrical signal can mimic this with a varying voltage between high and low values

Recording Audio data

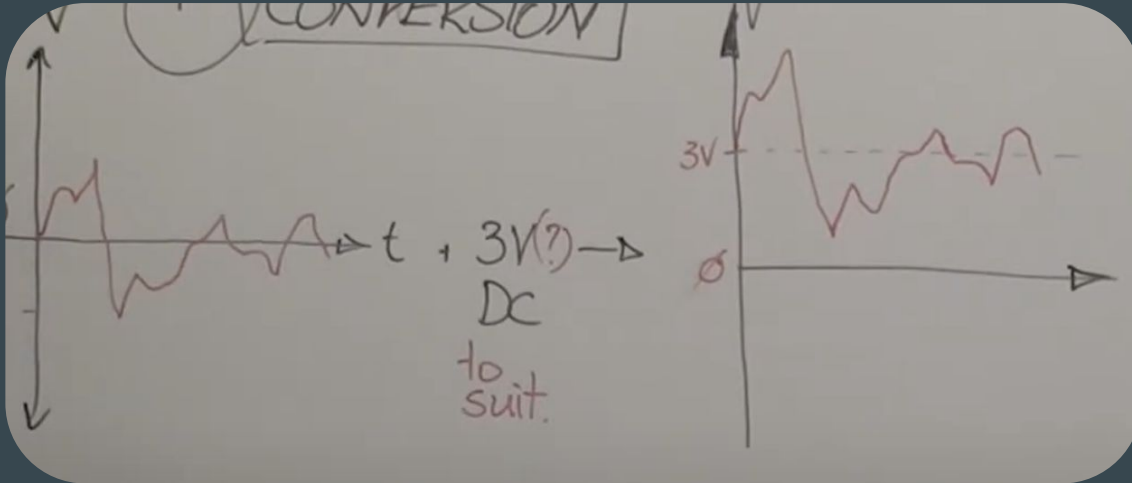
The voltage of the recorded signal relates to the amplitude of the original wave. Mic's produce AC waves, where a negative voltage (backwards current) represents the low points of the wave



Recording Audio data

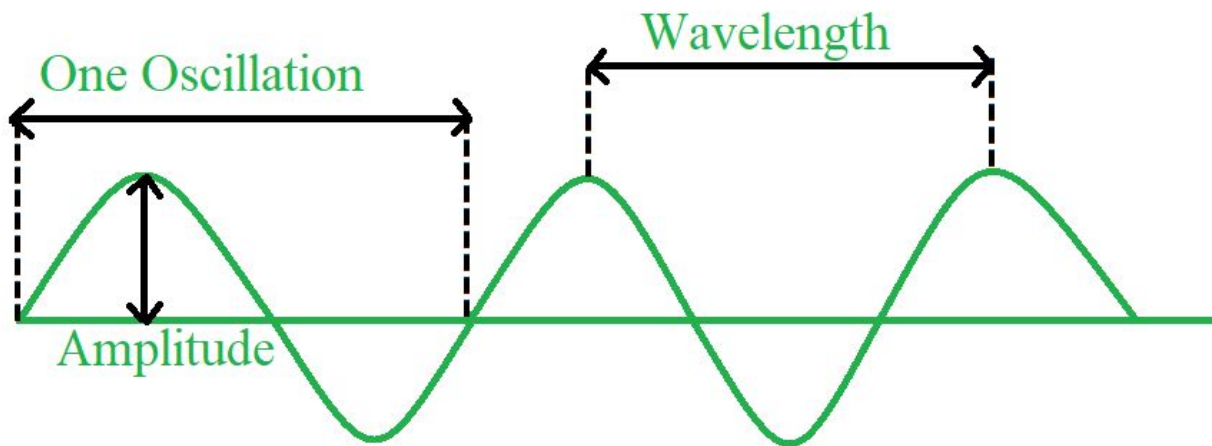
We are using the MAX4466 microphone, which has a built in amplifier. It supports a bandwidth of up to 600 kHz. When no audio is recorded, a value of $V_{CC}/2$ is output(not really, but it's meant to apparently). The mic shifts up the signal, which is necessary for the microprocessor and largely does not affect the audio.

Microprocessors don't like negative voltages



Cleaning Audio data

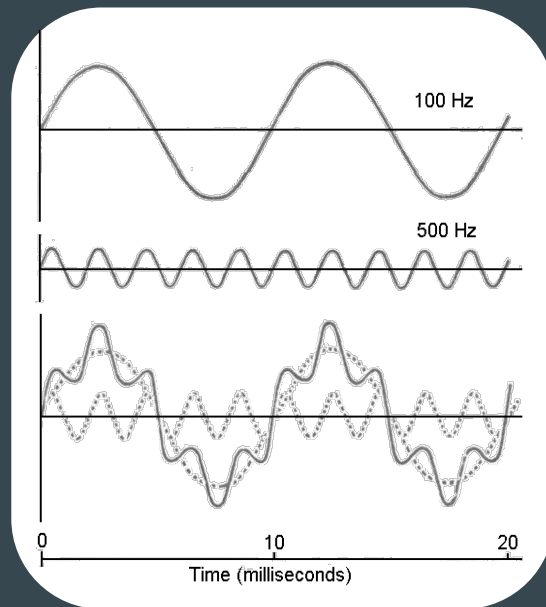
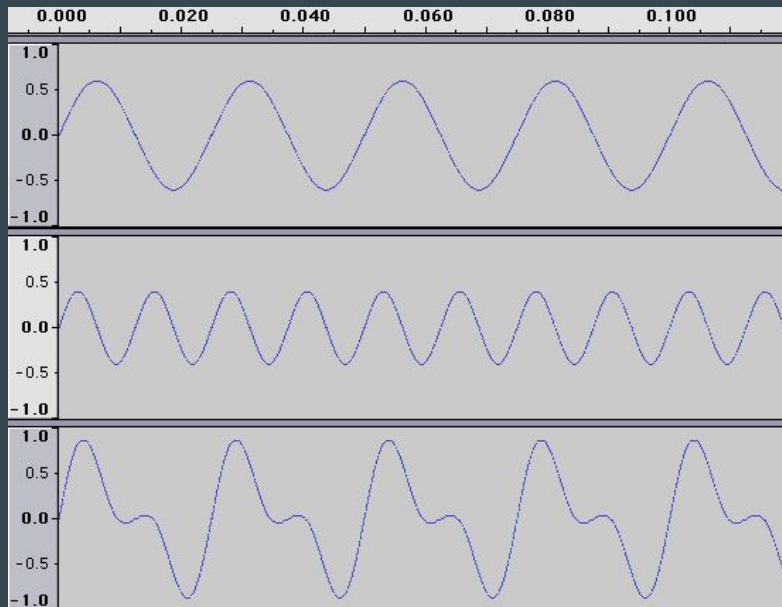
On top of amplitude (high pressure vs low pressure) waves also have the property of frequency, which is the number of ups and downs in a second



Frequency = Number of Oscillation per second

Cleaning Audio data

Any signal can be said to be the sum of a series of sine waves, where the frequency of the final signal is then the frequency of the highest input wave



Cleaning Audio data

According to WIKI voice range is roughly from 300 - 3000 Hz. As such to capture voice, you would want to record up to 4kHz, which requires sampling at 8kHz(we'll get to that in a bit)

speech.

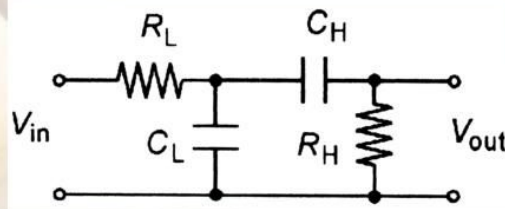
Frequency band [\[edit \]](#)

In [telephony](#), the usable [voice](#) frequency band ranges from approximately 300 to 3400 [Hz](#).^[2] It is for this reason that the [ultra low frequency](#) band of the [electromagnetic spectrum](#) between 300 and 3000 Hz is also referred to as *voice frequency*, being the electromagnetic energy that represents acoustic energy at [baseband](#). The [bandwidth](#) allocated for a single [voice-frequency transmission channel](#) is usually 4 kHz, including [guard bands](#),^[2] allowing a [sampling rate](#) of 8 kHz to be used as the basis of the [pulse-code modulation](#) system used for the digital [PSTN](#). Per the [Nyquist–Shannon sampling theorem](#), the sampling frequency (8 kHz) must be at least twice the highest component of the voice frequency via appropriate filtering prior to sampling at discrete times (4 kHz) for effective reconstruction of the voice signal.

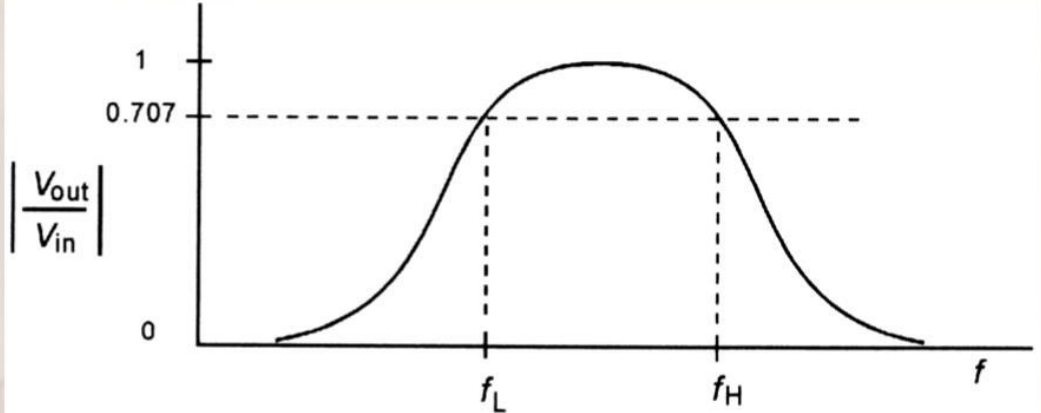
Name	Range (Hz)
Narrowband	300–3,400
Wideband	50–7,000
Superwideband	50–14,000
Fullband	20–20,000

Cleaning Audio Data

We use a passive band pass RC filter to reduce the amplitude of unwanted frequencies



**Band-pass
RC filter**



Cleaning Audio data

Calculating Band pass filter

Break frequency (bass cut) = 250Hz

$$F = 1/2\pi R_i C_i - R_i = 1/(2 * \pi * 250 * 1 * 10^{-6}) = 636.6\Omega \approx 600\Omega$$

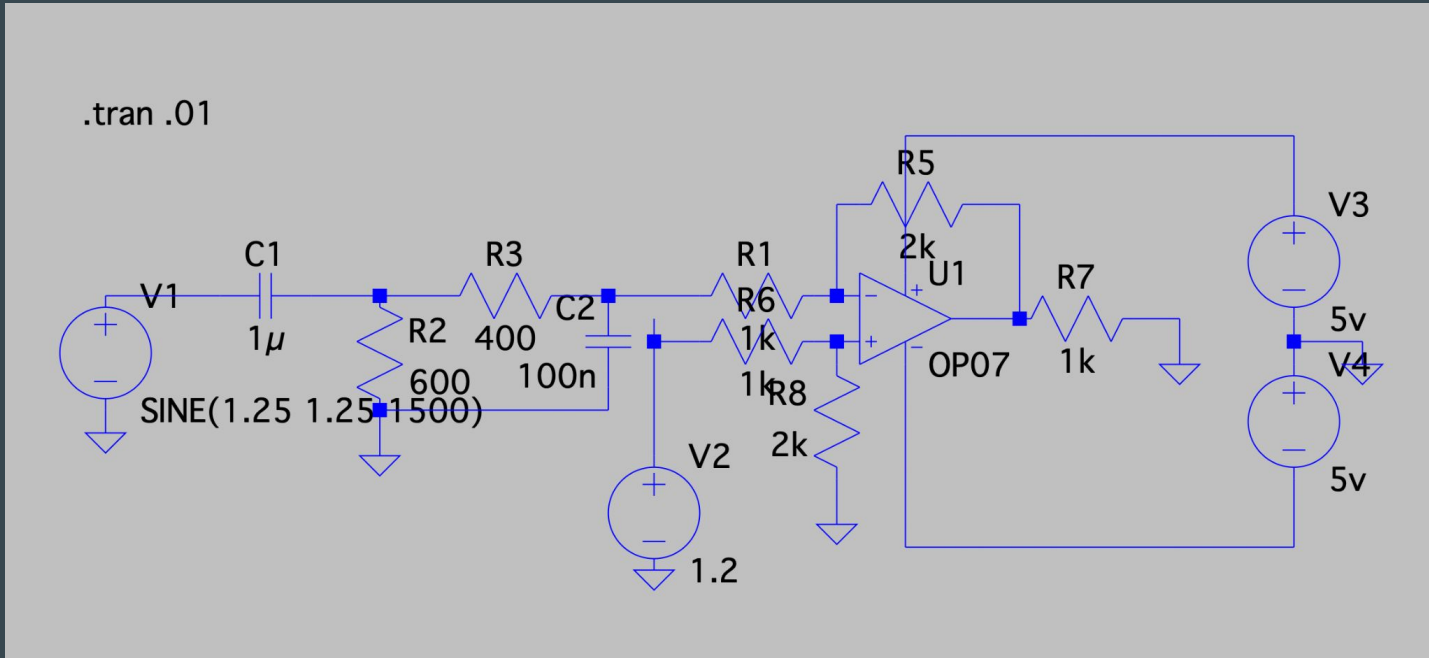
Break frequency (treble cut) = 3500Hz

$$F = 1/2\pi R_f C_f - R_f = 1/(2 * \pi * 3500 * 100 * 10^{-9}) = 454.7\Omega \approx 440\Omega$$

This reduces the mic signal from 0-2.5V to roughly -0.8 to 0.8V, so we need to shift up the signal and amplifier it, which we can do with an op amp

Cleaning Audio data

Since we are using a passive RC filter the amplitude of the outputted wave is weaker than the original signal and shifted down, which is a problem for the microcontroller.

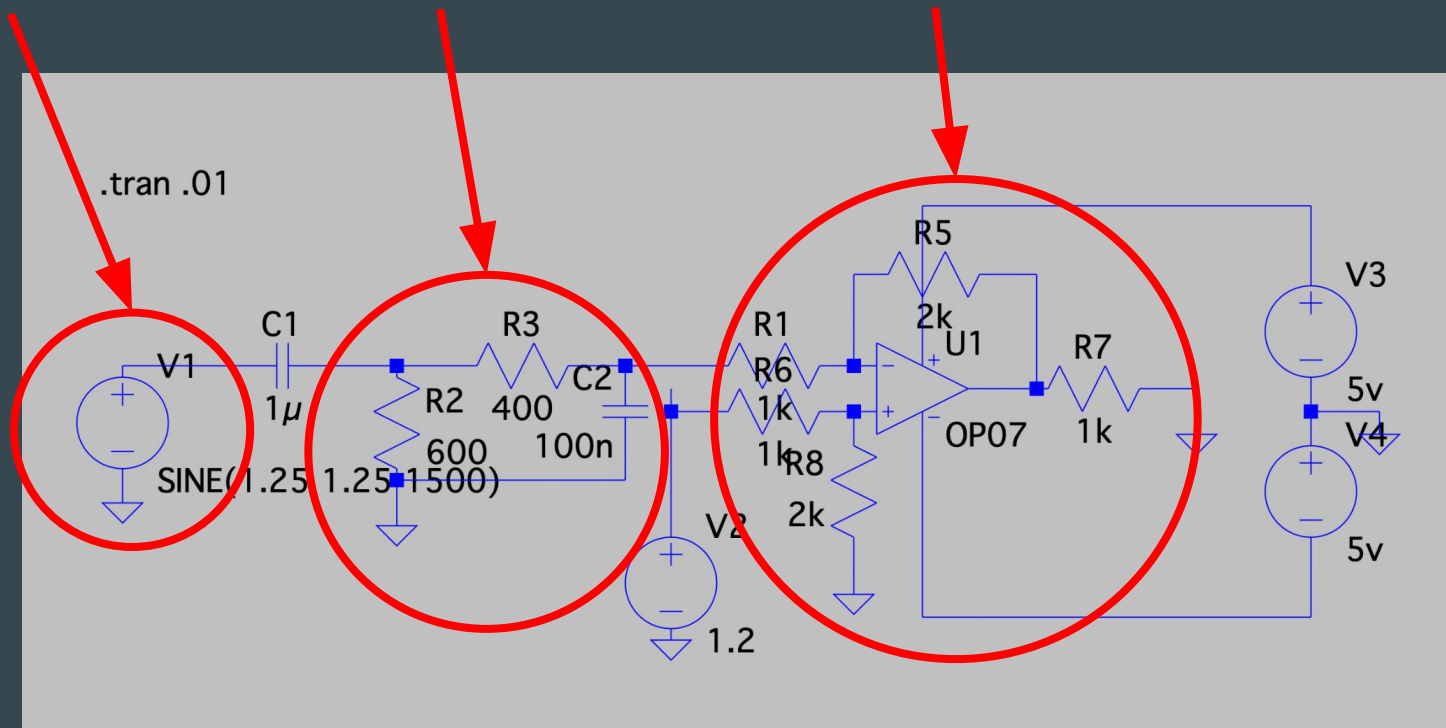


Cleaning Audio data

Mic

Filter

Amp

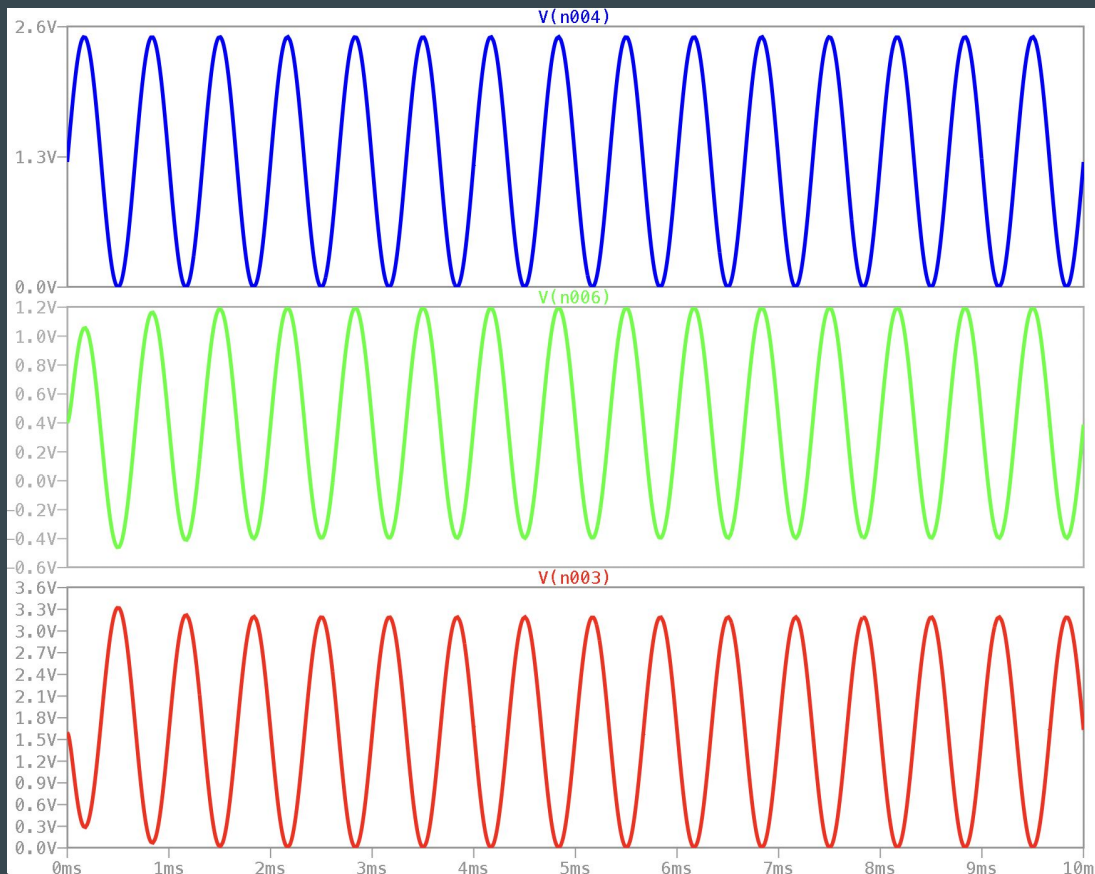


Cleaning Audio data

Original signal(1.5kHz)
Signal between 0 - 2.5V

Post filter
Signal between -0.4 - 1.2V

Post Amp
Signal between 0 - 3.3V
(Desired range)

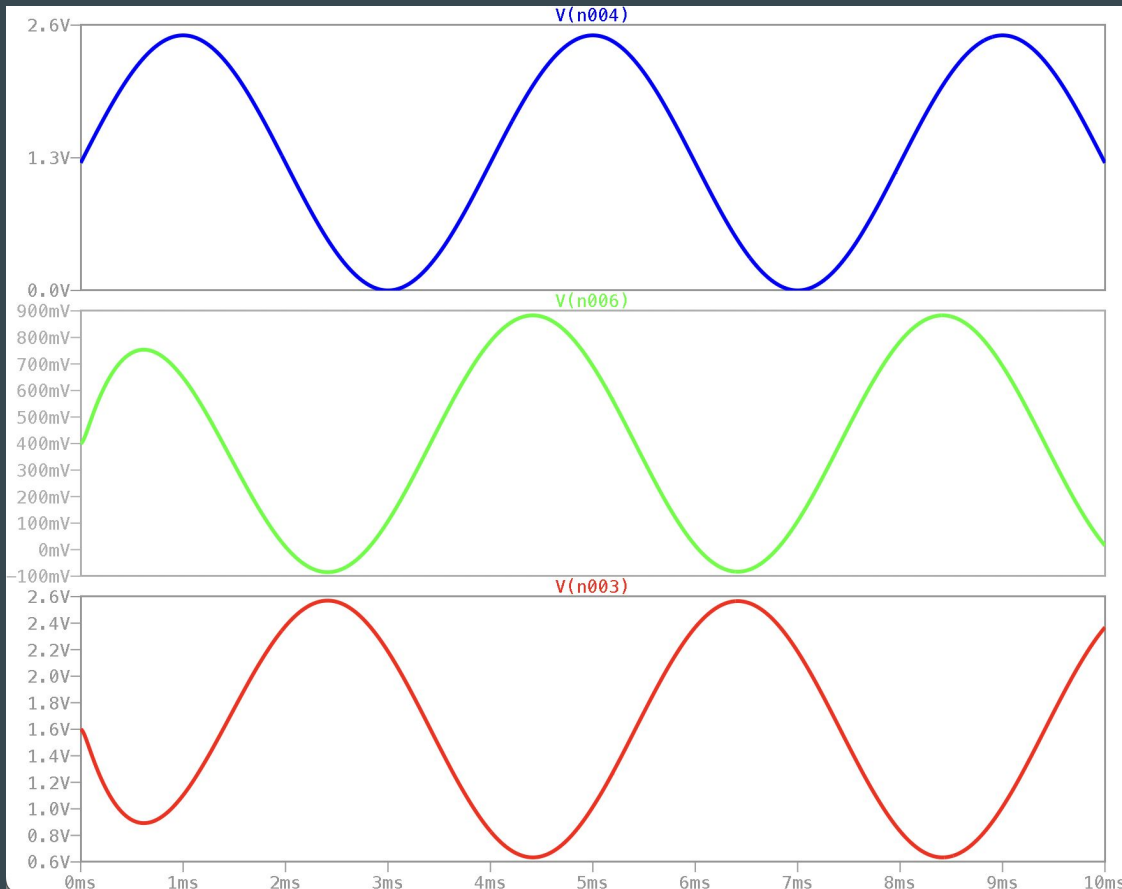


Cleaning Audio data

Original signal(250Hz)

Post filter -0.1 - 0.8V

Post Amp 0.6 - 2.6V

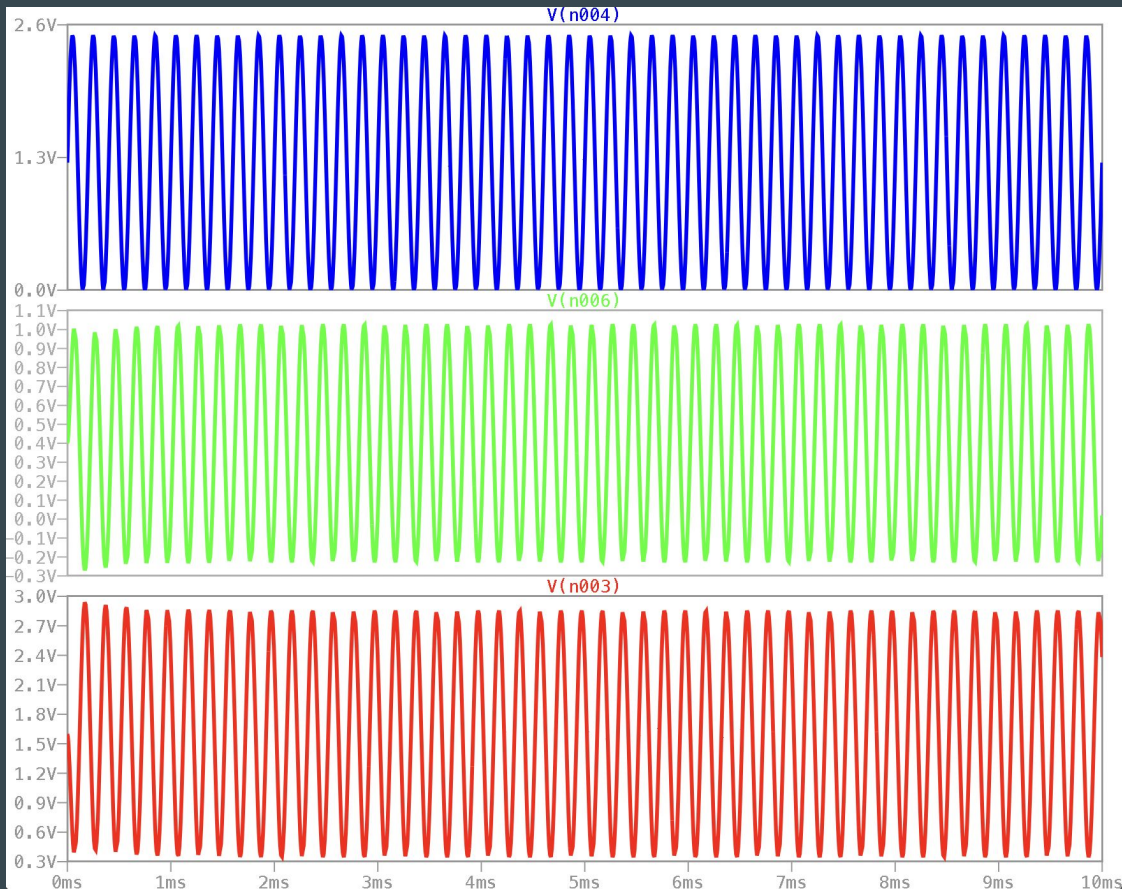


Cleaning Audio data

Original signal(5kHz)

Post filter -0.3 - 1.0V

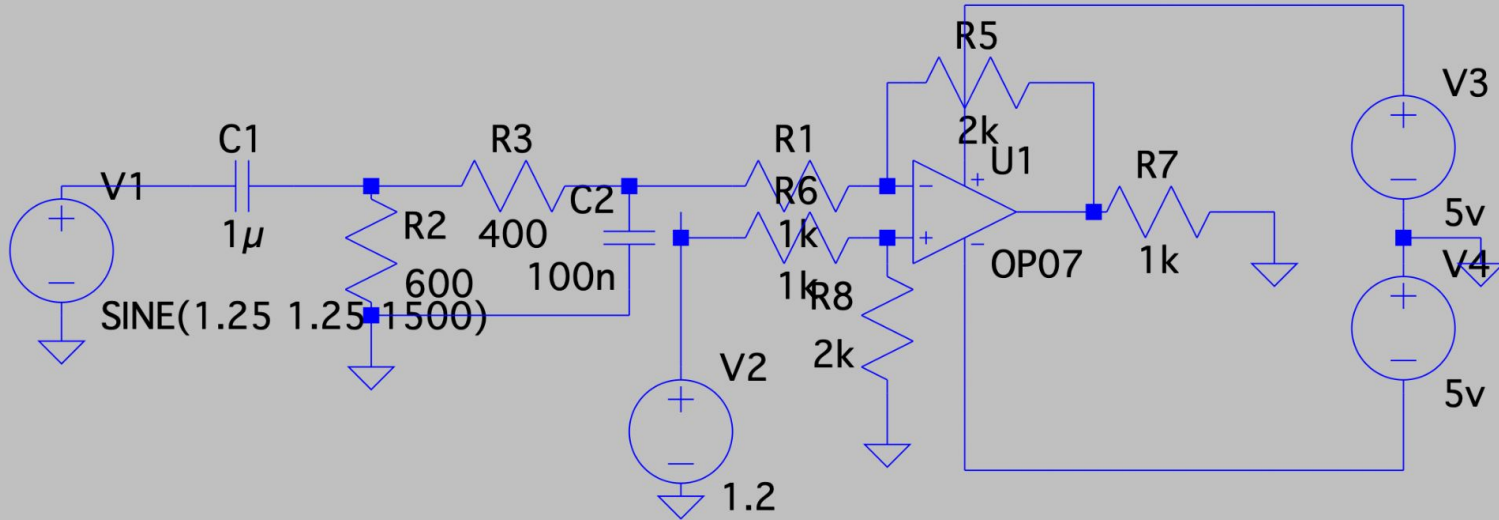
Post Amp 0.3 - 2.7V



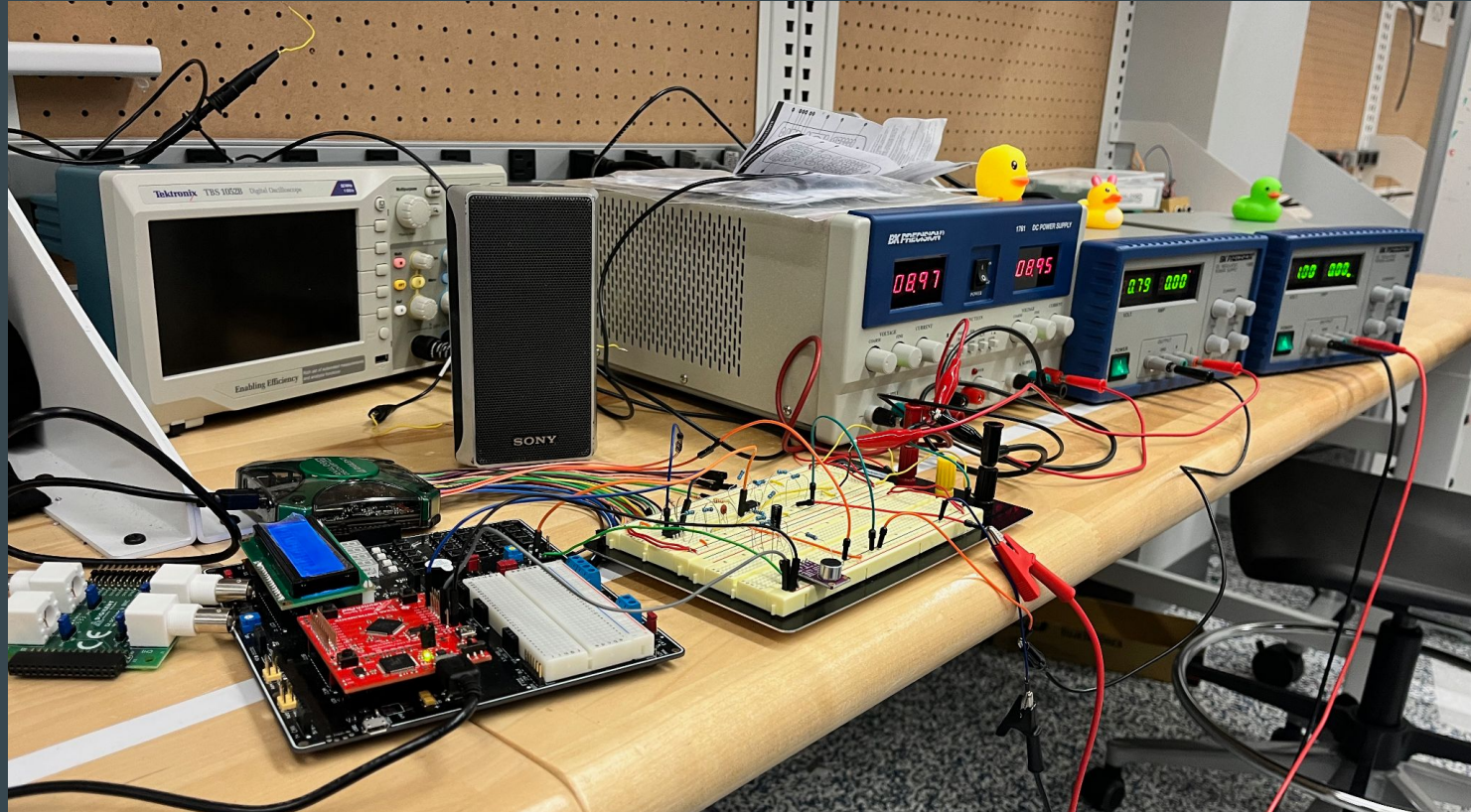
Cleaning Audio data

The amp inverts the signal, shifts it up and amplifies it (more on op amp stuff later)

.tran .01



The extremely simple setup



How does it do?

Example audio output from different parts of the circuit recorded at 50kHz

Audio straight from Mic



Mic Audio amplified and cleaned digitally



Audio After filter and amp



Sample Rate

What sample rate to choose?

According to the Nyquist–Shannon sampling theorem:

Theorem — If a function $x(t)$ contains no frequencies higher than B hertz, then it can be completely determined from its ordinates at a sequence of points spaced less than $1/(2B)$ seconds apart.

So if we wanted to record up to 4kHzs, we would just need to sample at 8Khz, but the industry standard is 41kHz for most audio.

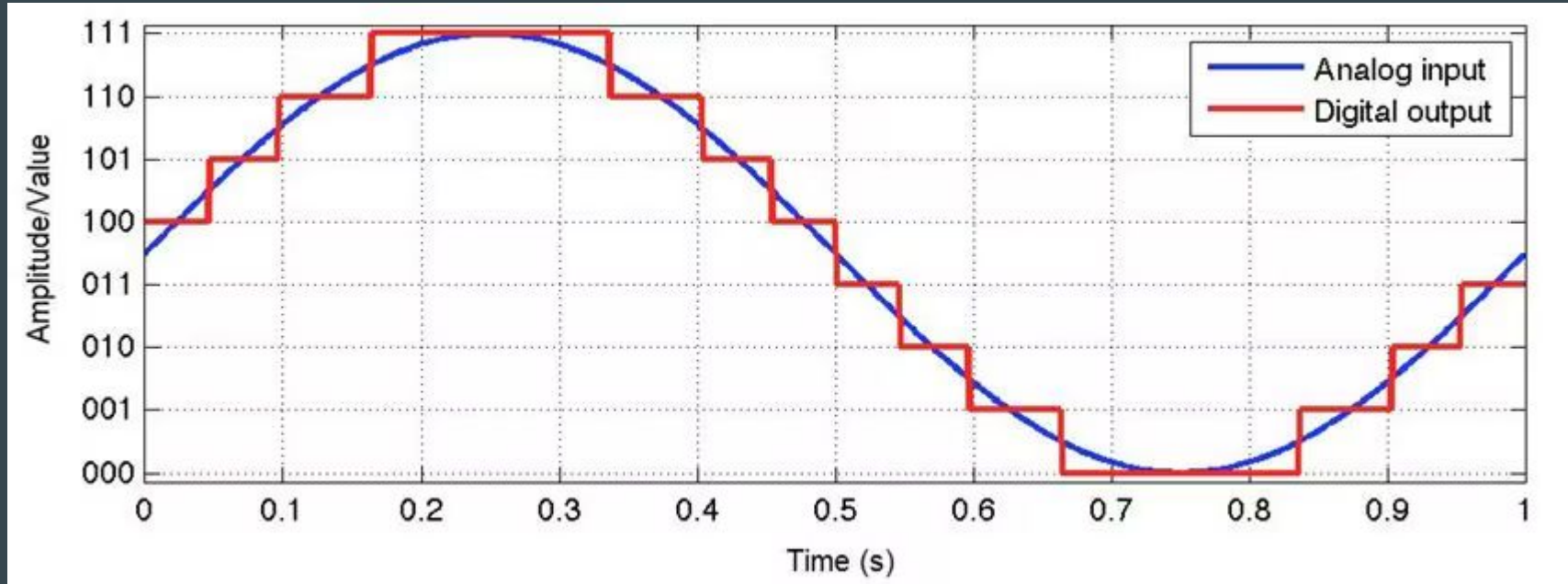
Calculating Timer

Tiva runs at 16Mhz, so the timer counter to 16M every second. If we want to the timer to interrupt at 8Khz.

$$16M * 1/8K = 2000 = 0xFA0$$

Analog to Digital Convertor

We use the Tiva's ADC to convert the analog signal to a digital signal

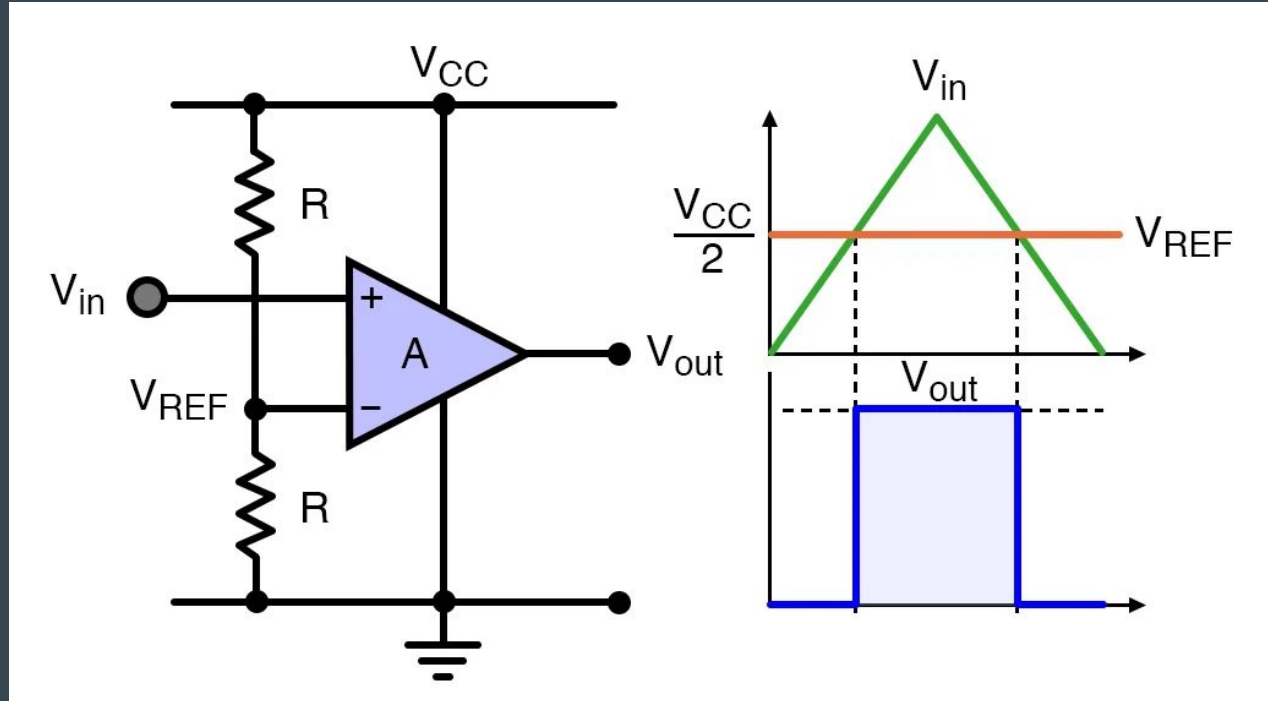


Analog to Digital Convertor

The ADC supports 12-bit conversion resolution and handles input from 0 - 3.3V, and has a maximum sample rate of one million samples/second

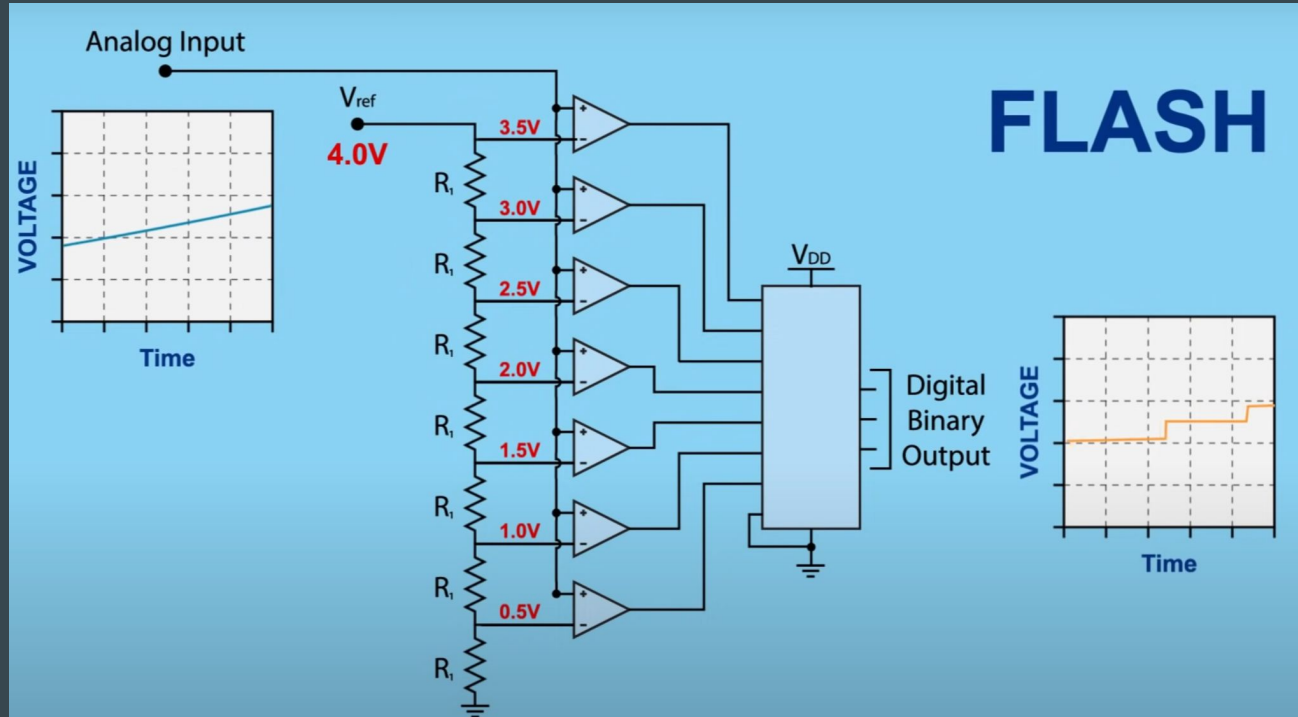
Analog to Digital Converter

How do ADC's work? There are many different versions, but one design uses comparators.



Analog to Digital Convertor

Comparators are then used in parallel to instantly check the input signal voltage level. Below is a 3 bit ADC(the tiva probably doesn't use this design)



Transmitting data:

Data Rates

Baud is the common unit of measurement for transmission speed, which measures symbols per second or pulses per second.

In this case, a symbols not defined as a character but as an individual transmission, so for us(and most cases with computers) baud = bits / second.

So if we are recording at 8kHz, that means every second we record 8000 12-bit samples, which is a total of 96,000 bit/second.

However we would basically be required to send 16-bits(2-bytes of data) per sample so we really need 128,000 baud.

Data Rates

Common standard baud rates are 115,200 baud and 9600 baud which are less than the required 128,000 baud mentioned earlier.

As such some slower transmission methods like IC2 weren't usable, which made bluetooth communication not usable :(

Data Rates

However if we only send the top 8 bits of the 12 bit data we can send data out from the Tiva to a computer using Uart ($8\text{-bits} * 8000 \text{ samples a second} = 64,000 \text{ baud}$)



Data Rates

The Tiva's UART has a max speed of 312500 baud(I think?)

The TM4C123GH6PM controller includes eight Universal Asynchronous Receiver/Transmitter (UART) with the following features:

- Programmable baud-rate generator allowing speeds up to 5 Mbps for regular speed (divide by 16) and 10 Mbps for high speed (divide by 8)
- Separate 16x8 transmit (TX) and receive (RX) FIFOs to reduce CPU interrupt service loading

I tried increasing the speed but things went weird and I'm not sure what the problem was

The first thing to consider when programming the UART is the baud-rate divisor (BRD), because the **UARTIBRD** and **UARTFBRD** registers must be written before the **UARTLCRH** register. Using the equation described in "Baud-Rate Generation" on page 896, the BRD can be calculated:

$$\text{BRD} = 20,000,000 / (16 * 115,200) = 10.8507$$

which means that the **DIVINT** field of the **UARTIBRD** register (see page 914) should be set to 10 decimal or 0xA. The value to be loaded into the **UARTFBRD** register (see page 915) is calculated by the equation:

$$\text{UARTFBRD}[\text{DIVFRAC}] = \text{integer}(0.8507 * 64 + 0.5) = 54$$

Outputting Audio:

Outputting Audio

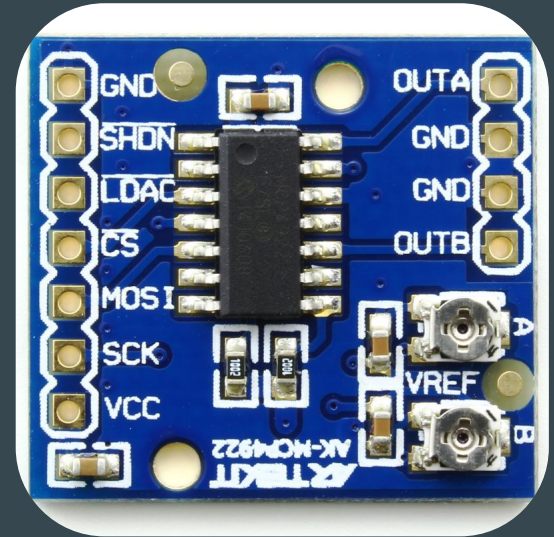
Originally the plan was to use the DAC on the Alice board which is a MCP4922. However it was too slow for our purposes.

In theory it could be fast enough since the SPI Interface has 20 MHz Clock Support, but we use IC2 which I believe has a default baud of 9600.

It also has a settling time of $4.5\ \mu\text{s}$, which should be fast for 8000Hz as $1/8000 = 0.000125\text{s} = 125\mu\text{s}$

4.9 Dual Digital-to-Analog Converters (DACs)

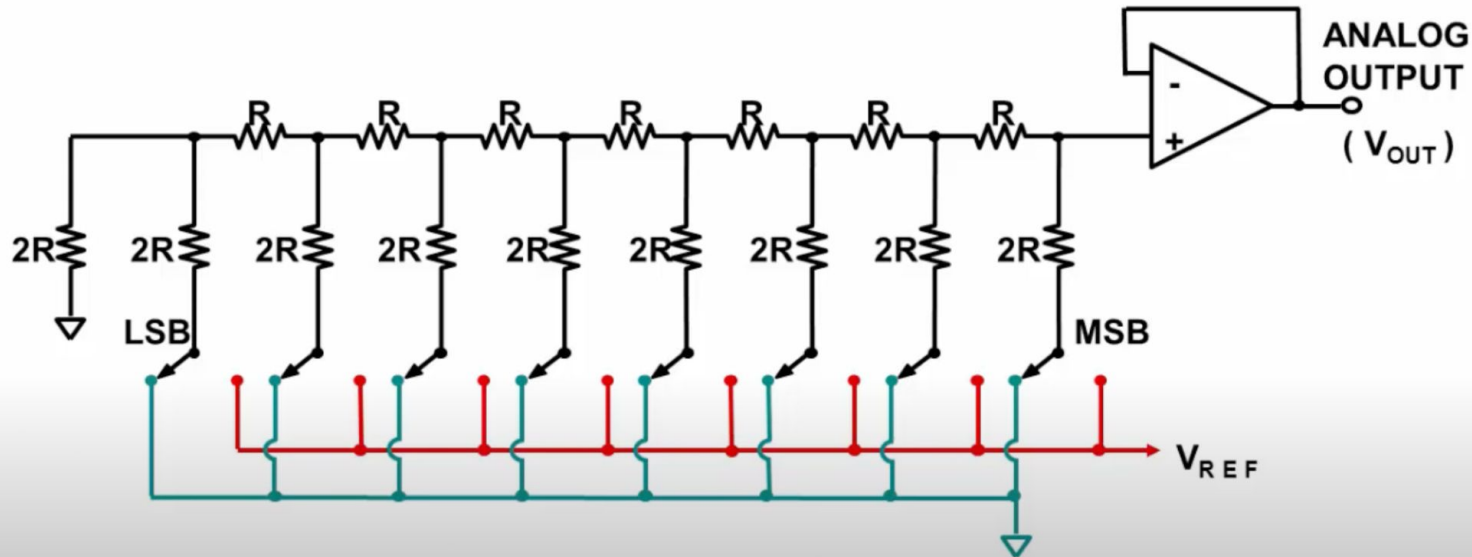
The MCP4922, a 2-ch, 12-bit SPI based DAC is installed for learning SPI communication. It converts a digital binary code to an analog signal so a program can generate different waveforms from the DAC.



Outputting Audio

As such looked into how DAC's worked and the String R-2R was simple enough to set up

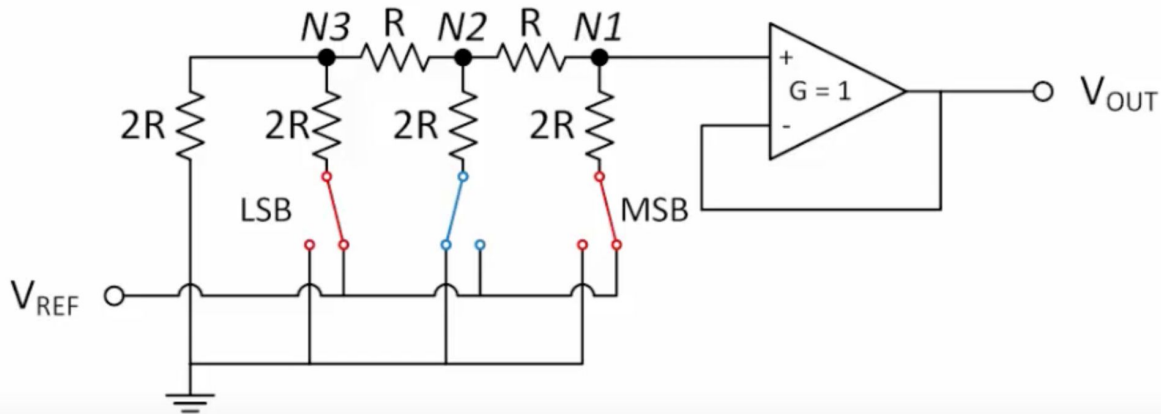
R-2R Back DAC



Outputting Audio

Every input is $2R$ from ground (Thevenin equivalent magic) and has a sum of resistors which each half the input value

Putting it together

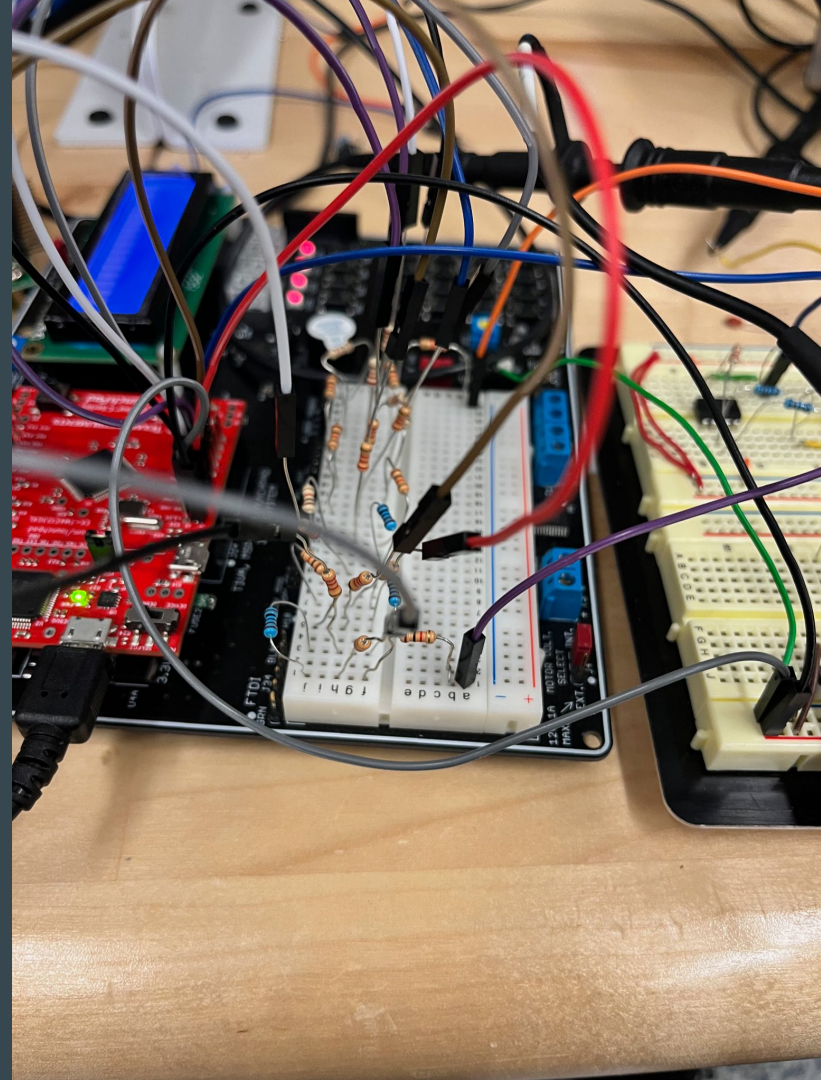


3-bit ladder:
$$V_{OUT} = \frac{V_1}{2} + \frac{V_2}{4} + \frac{V_3}{8}$$

Outputting Audio

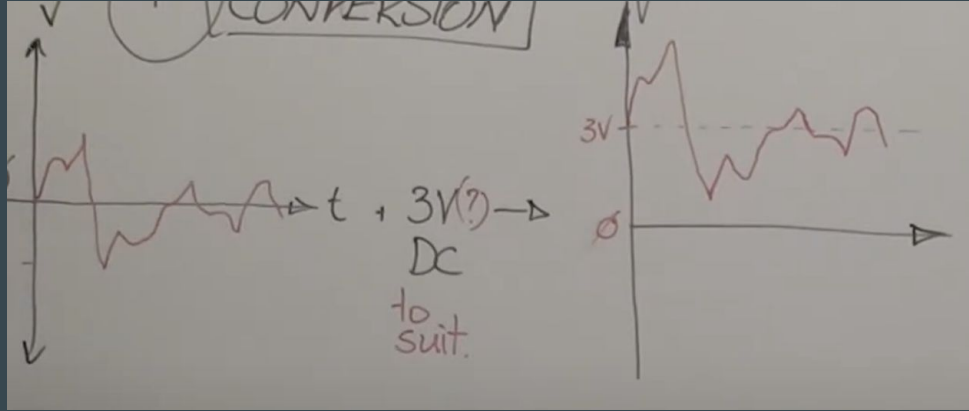
This Dac uses Ports B,C,D for outputting the 12 bit values.

Each pin should output 5V's but it ends being around 2.345V and the full range of the DAC is iffy but it kinda works

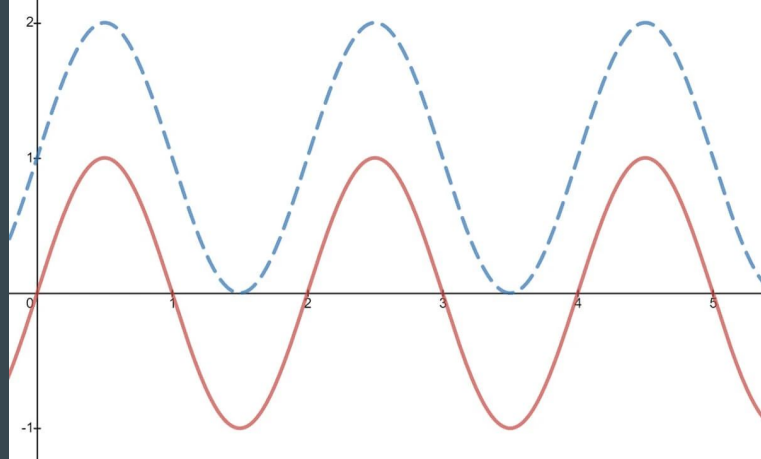


Outputting Audio

Remember this from before

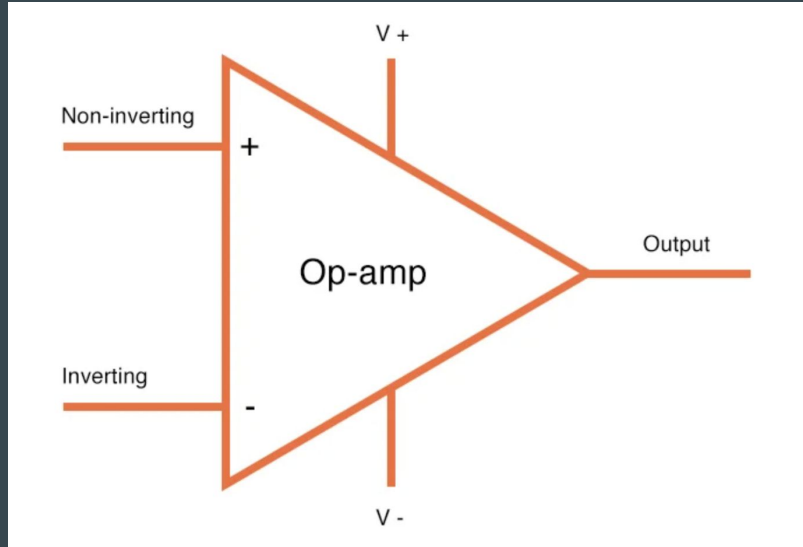


We need to do that but in reverse, as DAC's produce positive voltages, but the original signal is between positive and negative voltages



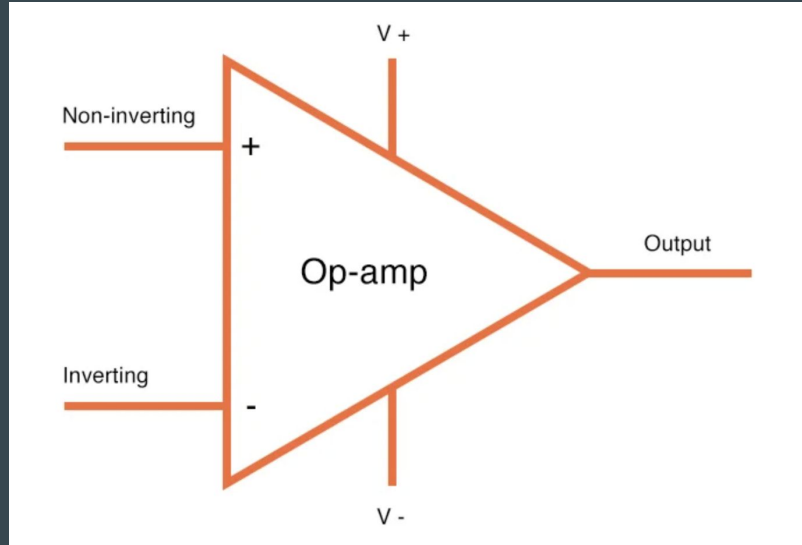
Outputting Audio

To apply a negative DC bias we can use an Op-amp. Op-amps take two inputs and produce a output



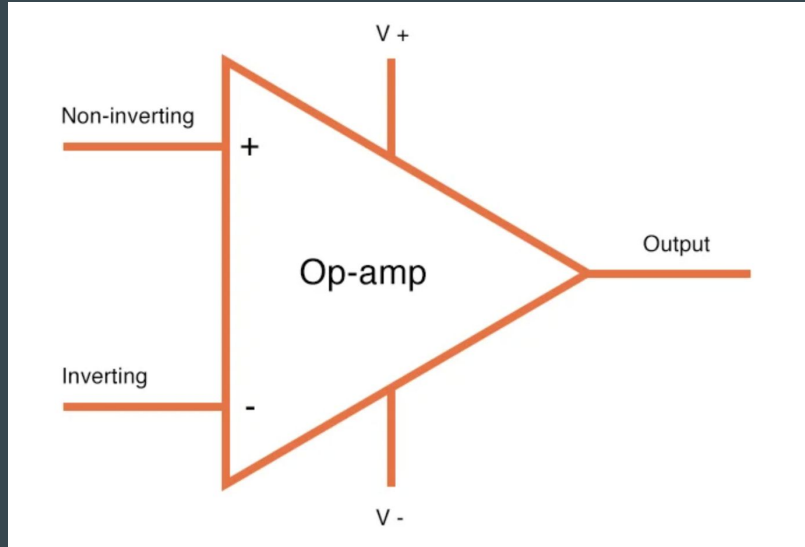
Outputting Audio

Non-inverting input passes through unchanged



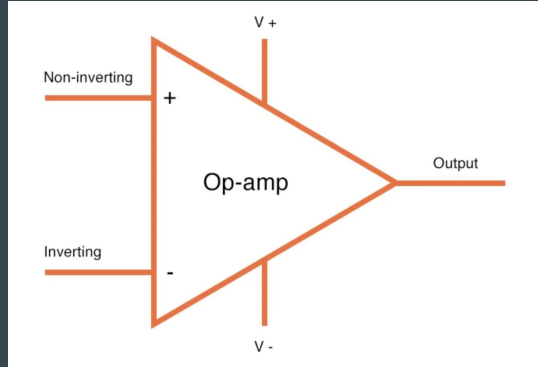
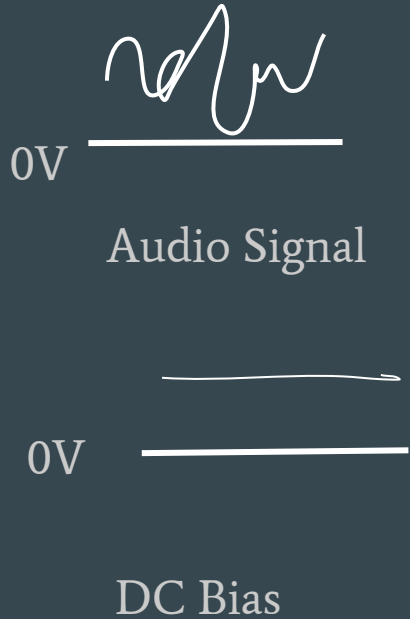
Outputting Audio

Inverting input gets shifted 180



Outputting Audio

The Op Amp then takes those two inputs adds them together



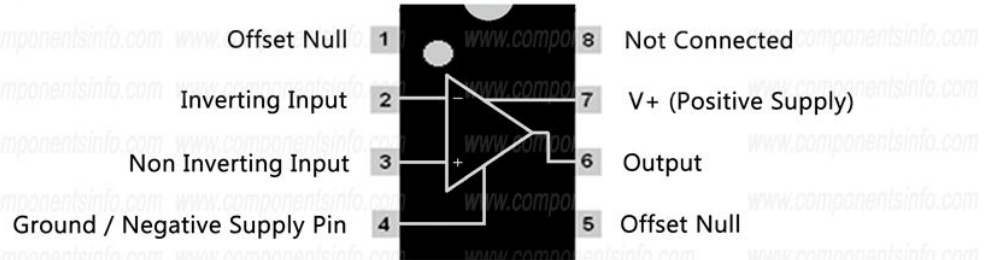
Outputting Audio

We are using the LM741

LM741 IC Pinout



LM741 DETAILED PIN DESCRIPTION



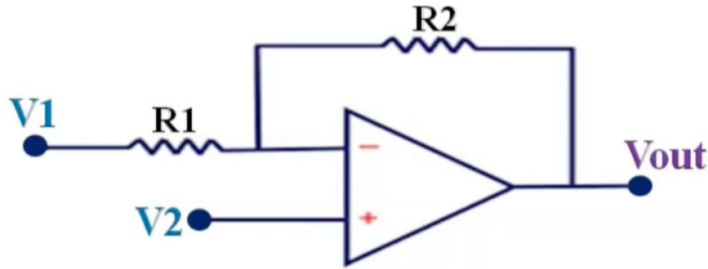
www.componentsinfo.com

Electronics Components Uses, Features, Pinouts, Equivalents, Applications & More...

Outputting Audio

We are using the LM741

OP-Amp: Differential Amplifier



$$\left(1 + \frac{R2}{R1}\right) = \left(\frac{R1+R2}{R1}\right)$$

$$\left(\frac{R1+R2}{R1}\right) \times \left(\frac{R2}{R1+R2}\right) = \left(\frac{R2}{R1}\right)$$

$$V_{out} = V_{out1} + V_{out2}$$

$$V_{out} = \left(-\frac{R2}{R1}\right) \times V1 + \left(1 + \frac{R2}{R1}\right) \times V2$$

If gain of both configurations are equal,

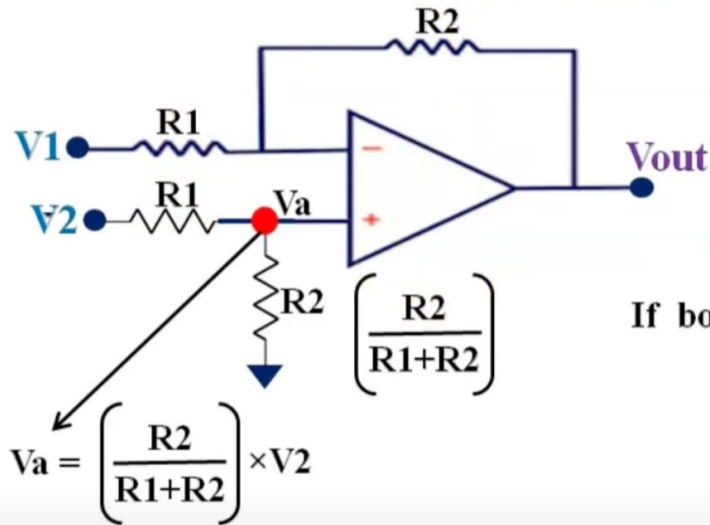
$$\left(-\frac{R2}{R1}\right) = \left(1 + \frac{R2}{R1}\right)$$

$$V_{out} = V_{out2} - V_{out1}$$

Outputting Audio

We are using the LM741

OP-Amp: Differential Amplifier



$$\text{If } V1=0, \quad V_{out 2} = \left(1 + \frac{R2}{R1} \right) \times V_a$$

$$V_{out 2} = \left(\frac{R1+R2}{R1} \right) \left(\frac{R2}{R1+R2} \right) \times V2 = \left(\frac{R2}{R1} \right) \times V2$$

If both $V1$ & $V2$ are present

$$V_{out} = \left(-\frac{R2}{R1} \right) \times V1 + \left(\frac{R2}{R1} \right) \times V2$$
$$= \left(\frac{R2}{R1} \right) \times (V2 - V1)$$

If all resistors are of equal value " $R1=R2$ "

Final result

Data Rates

8kHz sample rate



30kHz sample rate



Old audio sample using the Alice board DAC, sampled at 20MHz (this is a 40MB file and breaks google)